```
AssertionError: 42 != 200

----------------------------------------------------------------------
Ran 1 test in 0.005s

FAILED (failures=1)
Destroying test database for alias 'default'...
```

Test assert methods raise exceptions if their conditions don't get met. In this case, `.assertEqual()` has decided that `42 != 200`. Who am I to argue? Since assertions raise exceptions, you get a stack trace indicating where the failure happened. At the bottom of the output, you still see how many tests got run but also how many tests failed.

> **Django likes trailing slashes**
>
> Django expects a URL to end with a slash (/). If a URL does not end in a slash, Django issues a permanent redirect (HTTP `301`) with the correction. Users don't notice this, as the browser takes care of it. If you forget the trailing slash in your test code, you'll get the wrong status code in response. Make sure all your URLs in your test suite end in a slash, and you'll be fine.

That's your first unit test for a view. One down, a little over 20 more to go. Don't forget to change `42` back to `200`.

8.3 Testing with the database

The `credits()` view is pretty much the simplest view you can write. Testing more complex views requires a bit more effort, especially if you're interacting with the ORM. The Django test runner creates a new test database each time it starts up. The good news is you don't have to worry about your tests modifying your site's data. The bad news is your test environment has an empty database.

When run, each test gets wrapped in a database transaction, which is a way of atomically grouping a series of actions. When the test method completes, Django rolls back the transaction, undoing anything you did in the test method. This might sound like a problem, but it is actually a good thing: the tests can be run in any order, each starting with the exact same state.

You likely want some things in your database to use with the tests, and fortunately, you don't have to do this for every single test call. The Python `unittest` class has a method called `.setUp()` that gets invoked just before each test gets run. This is the perfect place to create your ORM objects as class members.

Let's write a test for the `musician()` view in the Bands app. You'll recall this view displays details for the `Musician Model` passed in as an argument. Figure 8.3 shows the setting up and testing of this view.

In the diagram, you see the first thing the test needs to do is create a `Musician` object in the test database; it does this inside the `.setUp()` method, keeping it as an attribute of the test case. This makes your `Musician` object available in all tests. Next,